SEARCH 22.5: Spectral Embedding Archive for Rational Crease Pattern Hyperspace

Brandon Wong

May 7, 2026

Abstract

This report presents SEARCH (Spectral Embedding Archive for Rational Crease Pattern Hyperspace), an inverted design methodology for computational generation of 22.5° crease patterns. Instead of solving a highly intractible nonlinear mixed-integer optimization problem for every user input, we precompute a massive database of millions of valid uniaxial 22.5° crease patterns. At runtime, the system then searches the database for the precomputed crease pattern whose uniaxial tree most closely matches the user's desired structure. The database uses laplacian eigenvalues of the uniaxial trees as embeddings to index structural similarity (*spectral embedding archive*). Furthermore, floating point errors due to 22.5's inherent $\sqrt{2}$ computations are circumvented by projecting vertices onto a 4D rational basis (*rational crease pattern hyperspace*). By shifting the computational burden to an exhaustive precomputation phase, user design is reduced to a FAISS query, which runs in constant time relative to input complexity. This report details the mathematical architecture of the database generation pipeline, which is divided into three phases: (I) Topology Enumeration, (II) Tiling optimization for foldability, and (III) Crease Pattern generation and tree extraction.

1 Introduction

One of the biggest unsolved problems in origami design is the ability to systematically design models in the 22.5° angle system, in which creases meet at angles of multiples of 22.5°. Many of the oldest known traditional origami designs use this angle system (including the fish, crane, and frog), as do some of the most popular modern-day designs, yet there remain no known mathematical methods to systematically design models within these angle constraints. Instead, most 22.5° origami designers rely on trial and error combined with an experienced artist's intuition.

In general, representational origami designs can be categorized into two categories: uniaxial and non-uniaxial. Uniaxial designs are those whose structure can be represented by a tree of flaps and rivers. Forward design with uniaxial methods, including circle packing, box pleating, and hex pleating, have been well-studied. In these systematic procedures, a tree is converted into a packing of circles, squares, or hexagons by formulating the problem an optimization problem defined by non-linear non-intersection constraints. Then, a crease pattern can be obtained from the packing by linear time deterministic procedures

It is theoretically possible to inject rigid integer constraints into the nonlinear solver to enforce the angle constraints of the 22.5° system. However, even if this massive MINLP problem is solved and a valid packing is found, there is no known way to convert the packing into a flat foldable crease pattern due to a phenomenon known as "dense bouncing". Unlike box pleating or hex pleating with rational quantized lengths, 22.5° angles generate lengths involving $\sqrt{2}$, for which no amount of subdivision will ever perfectly line up with a rational length, effectively requiring an infinite number of creases to resolve the packing. And unlike circle packing which solves this by placing creases at specific angles, 22.5° crease patterns are by definition restricted to creases at multiples of 22.5°.

Conversely, generating a flat foldable crease pattern from a predefined tiling of 22.5° molecules is computationally straightforward through the use of straight skeletons. However, the inverse design of a tiling to acheive a desired tree has no known solution—in fact, prior to this project, there were no known implementations of the forward computation of extracting a tree from a tiling.

Therefore, we we exploit the computational tractibility of tilings to enumerate a massive database of valid 22.5° tilings, and then extract their underlying trees retroactively. By indexing the database using spectral embeddings of the uniaxial trees, we can approximate the user's inverse design problem (Tree → Crease Pattern) in constant time by simply searching for the closest spectral embedding in the database and returning its corresponding crease pattern.

1.1 Packing, Tiling, and Molecules

1.2 Definitions and Nomenclature

To document the SEARCH 22.5 data pipeline, we define the following levels of abstraction across our architecture:

- **Topology:** An unweighted planar graph representing the angles (multiples of 45°) and connective relationships of axial creases, prior to assigning specific lengths.
- **Tiling:** A metric realization of a topology, or in other words, a planar graph of axial creases with assigned lengths.
- **Crease Pattern:** A planar graph of creases that includes the axial creases of the tiling, ridge creases formed by the straight skeleton of the tiling polygons, and any additional creases (hinges) required to fold the model flat.
- **Fold/Folded form:** The folded state of a crease pattern, defined by an unordered set of faces and their connectivity in 2d space. In this report, we neglect layer ordering and self intersection.
- **Uniaxial Tree:** The projection of the folded form onto some axis, resulting in a tree structure of flaps and rivers representing the stick figure of the folded form.

2 Vertex and Crease Level Architecture

The first major obstacle in the 22.5° system is representing vertices. In conventional crease pattern software, points are stored as floating-point Cartesian pairs. Because 22.5° triangles immediately generate irrational lengths, we encounter severe numerical precision issues during raycasting and intersection checking if we attempt to use this conventional data structure.

We observe that all Cartesian coordinates in the 22.5° angle system can be represented analytically in the form $(A_x + B_x\sqrt{2}, A_y + B_y\sqrt{2})$, where A_x, B_x, A_y, B_y are rational numbers. This suggests that we can represent points as a tuple of four fractions, avoiding numerical drift entirely by storing the vertex as a list of integers.

2.1 The 4D Hypercube Projection

It would be possible to directly convert the 2D Cartesian coordinate to the orthogonal 4D rational coordinate (A_x, B_x, A_y, B_y). However, we elect to use a projection that spaces the basis vectors 45 degrees apart from one another, reminiscent of projecting a 4D hypercube in 2D space. Rational 4D coordinates project uniquely to the 2D plane, and 2D 22.5° vertices "lift" uniquely into 4D.

[Figure Placeholder: A coordinate plane showing the 8-directional hypercube basis. The vertex (1,1,-1,1) is visually constructed by adding the basis vectors $\hat{x}, \hat{y}, -\hat{z}, \hat{w}$ tail-to-head.]

2.2 Crease Intersections and Exact Fractions

To intersect creases formed by vertices $\mathbf{v}_a$ and $\mathbf{v}_b$ at angles α and β, we utilize precomputed rational tangents. The intersection vertex $\mathbf{v}_c$ is calculated as:

$$\Delta i = (v_{b_x} - v_{a_x}) + (v_{b_y} - v_{a_y} - v_{b_w} + v_{a_w})\frac{\sqrt{2}}{2} \tag{1}$$

$$\Delta j = (v_{b_z} - v_{a_z}) + (v_{b_y} - v_{a_y} + v_{b_w} - v_{a_w})\frac{\sqrt{2}}{2} \tag{2}$$

$$\beta_i = \frac{\Delta j - \tan(\alpha)\Delta i}{\tan(\alpha) - \tan(\beta)} \tag{3}$$

To prevent exponential bit-depth growth during complex divisions, we implemented a custom, highly optimized C++ `Fraction` class exposed via Python bindings. This class strictly enforces immediate gcd simplification upon instantiation, ensuring the numerators and denominators remain safely within 64-bit integer limits across millions of operations.

3 Database Generation Phase I: Topology Enumeration

To build an exhaustive database of 22.5° crease patterns, we must enumerate the valid arrangement of axial creases that define the tilings. A topology defines the angles and connective relationships of axial creases, disregarding exact metric lengths. We exhaustively enumerate every valid topology on an $N \times N$ integer grid by modeling the problem as a Boolean Satisfiability (SAT) problem.

3.1 Grid Formulation and Boolean Variables

We embed a planar graph onto an $N \times N$ square grid. The boundary of the paper is fixed, comprising $4N$ predefined boundary segments.

Inside the grid, every possible orthogonal and diagonal segment connecting adjacent integer coordinates is treated as a potential axial crease. We assign a boolean decision variable $e \in \{0, 1\}$ to every internal edge. The total number of internal boolean variables for an $N \times N$ grid is precisely $4N^2 - 2N$. The SAT solver's task is to sort through the 2^{4N^2-2N} possible assignments to find unique topologies that satisfy the following rules. In the following formulation, we use $\overline{z}$ to denote the logical negation, $\cdot$ for logical AND, and $+$ for logical OR.

3.2 Local Z3 Geometric Constraints

To prune the inherently exponential search space, we restrict the Z3 solver using four core local constraints that physically validate the crease structure.

3.2.1 Planarity Constraint

An axial topology must be planar. The two diagonals within any single 1×1 would cross if both were present. Therefore, they are mutually exclusive. For every cell (x, y), we assert:

$$(\overline{d1_{x,y}} + \overline{d2_{x,y}}) \tag{4}$$

3.2.2 No Dangling Edges Constraint

A crease cannot terminate abruptly in the middle of a sheet of paper. Define the *degree* of a vertex as the number of active edges incident to it. Every internal vertex must have a degree either $= 0$ (inactive) or ≥ 2 (active). For every internal node, we assert that the sum of edges incident to it $\neq 1$.

3.2.3 No T-Junction Constraint

To prevent dense bouncing and other straight skeleton problems, we enforce a strict T-junction rule: a 180° continuous line cannot exist unless it is perpendicularly crossed by another line.

For every internal node $(x, y) \in [1, N-1]^2$, we evaluate its 8 potential neighbors in counter-clockwise order: Right (R), Up-Right (UR), Up (U), Up-Left (UL), Left (L), Down-Left (DL), Down (D), and Down-Right (DR).

We define four straight pairs: $(R, L), (U, D), (UR, DL), (UL, DR)$. The rule dictates that if *any* straight pair is continuously active, then *all* opposite pairs in that specific node must share identical boolean states to form a valid 4-way (or 8-way) crossing. In boolean logic, we assert:

$$\bigvee_{P \in \text{pairs}} (P_1 \wedge P_2) \implies \bigwedge_{P \in \text{pairs}} (P_1 = P_2) \tag{5}$$

3.2.4 Optional Symmetry Constraint

We can optionally enforce symmetry. For a requested symmetry (e.g., book symmetry across the vertical axis $x = N/2$, or diagonal symmetry across $y = x$), we apply the corresponding coordinate transformation to identify symmetric edges. For every internal edge $e = (u, v)$ and its transformed counterpart $e_{sym} = (T(u), T(v))$, we enforce:

$$e = e_{sym} \tag{6}$$

[Figure Placeholder: A 3×3 grid diagram illustrating the four local constraints. Specifically, show a cell with an invalid crossing diagonal marked with a red 'X', a valid 4-way intersection highlighting the T-junction rule, and a dotted line showing book symmetry reflecting edges from the left side to the right.]

3.3 Global Topological Constraints

Once Z3 yields a satisfying assignment based on local constraints, we extract the resulting graph $G = (V, E)$ and pass it through a Python-level global topology filter before accepting it into the database.

3.3.1 Connectivity Constraint

First, we require the graph to be a single connected component. Disconnected components represent polygons floating in space within other polygons, which causes severe problems during crease pattern generation. This is easily filtered using NetworkX's built-in connectivity check.

3.3.2 Unique Normals Constraint

Second, we validate the internal faces. We utilize a Doubly-Connected Edge List (DCEL) half-edge traversal to extract all faces of the graph: for every vertex v, its incident neighbors are sorted angularly using a standard atan2(dy, dx) function. Walking counter-clockwise around the leftmost turns allows us to systematically extract the closed loops that define the internal polygonal faces.

For each extracted face, we calculate the directional vector

$$\mathbf{d} = (\Delta x / \gcd(\Delta x, \Delta y), \Delta y / \gcd(\Delta x, \Delta y))$$

of every boundary segment. We strictly enforce that a valid topological face must consist of sides with unique directions (ignoring sequential collinear segments). While this allows concave polygons, it limits the number of sides to 8 and ensures they do not zig-zag or "gerrymander", which would also make crease pattern generation significantly more difficult.

With these local boolean constraints and global topology filters, the SAT solver is able to enumerate all valid topologies.

4 Database Generation Phase II: Metric Tiling Realization

The output of the topology enumeration phase is an unweighted planar graph. Although the vertices are temporarily built on an integer grid to guarantee 45° angled edges, the actual locations of the vertices need to be fully constrained to optimize for human-foldability of the crease pattern. This is, after all, the original purpose of 22.5° crease patterns over "easier" design methods.

If a topology contains n nodes, we theoretically possess $2n$ degrees of freedom in the Euclidean plane (moving each node in x and y). Our objective is to apply constraints until all degrees of freedom are removed, and the tiling is fully locked into a single crease pattern with fully defined references.

4.1 Basic Geometric Constraints

We begin by establishing the absolute requirements of the tiling. We first remove redundant degree-2 vertices from the topology.

4.1.1 Angle preservation constraints

Every edge in the initial grid topology is aligned to a multiple of 45°. As the vertices move to form the exact tiling, these edges must remain parallel to their original orientations. Let $\vec{u}$ and $\vec{v}$ be the position of two vertices connected by an edge, and $\hat{n} = (n_x, n_y)$ be the known normal vector of that edge. We enforce the following angular constraint for every edge:

$$(\vec{v} - \vec{u}) \perp \hat{n} \implies \hat{n} \cdot (\vec{v} - \vec{u}) = 0 \implies n_x v_x - n_x u_x + n_y v_y - n_y u_y = 0 \tag{7}$$

Note that this constraint is linear and homogeneous.

4.1.2 Boundary conditions

To prevent rigid-body translation and scaling, the bottom-left node is locked to the $(0, 0)$, and the top-right node is locked to $(1, 1)$. Note that the latter is an inhomogeneous constraint, which prevents the solver from collapsing all vertex coordinates to 0.

4.1.3 Symmetry constraints

If the topology was generated with a symmetry constraint, we apply the corresponding symmetry constraints to the coordinates. For example, for book symmetry across the vertical axis $x = N/2$, every pair of symmetric nodes u and u^* must satisfy:

$$u_x + u_x^* = 1 \quad \text{and} \quad u_y - u_y^* = 0 \tag{8}$$

Note that this introduces another inhomogeneous constraint. For diagonal symmetry across $y = x$, every pair of symmetric nodes u and u^* must satisfy:

$$u_x - u_y^* = 0 \quad \text{and} \quad u_y - u_x^* = 0 \tag{9}$$

We can then combine these linear constraints into a single matrix form $A\vec{x} = \vec{b}$, where $x = [x_1, y_1, x_2, y_2, \ldots, x_n, y_n]^T$ is a vector containing all node coordinates, each row of A corresponds to one of the aforementioned linear constraints, and $\vec{b}$ is a vector of constants (mostly zeros, except for the few inhomogeneous conditions).

The crease pattern is fully defined if and only if the matrix A has full rank, $\text{rank}(A) = 2n$. If the matrix is rank-deficient, then there are still degrees of freedom for vertices in the tiling to move without violating constraints. These motions are described by columns of the null space. To resolve these degrees of freedom, we need to inject additional constraints.

4.2 Quadruplet equidistance constraints

While only the basic geometric constraints are absolutely necessary, we can "spend" the remaining degrees of freedom to constrain the tiling into a crease pattern that is as "nice" as possible. It is difficult to quantify how "nice" a crease pattern is, but a good heuristic is to minimize the number of internal vertices generated by the straight skeletons of the polygons. This is because internal vertices of a straight skeleton are generally not flat foldable, so each internal vertex forces a propagation of creases that make the crease pattern increasingly messier.

Internal vertices in a straight skeleton appear where 3 or more edges are equidistant from a common point. We can "collapse" internal vertices together by forcing > 3 sides to be equidistant from a common point. Therefore, we define a **quadruplet** as a set of 4 edges (not necessarily contiguous) on a polygon, which we can constraint to be equidistant from a shared incircle.

Let $\hat{n}_1, \hat{n}_2, \hat{n}_3, \hat{n}_4$ be the inward-facing unit normal vectors of the four edges in the quadruplet. Let $\vec{v}_1, \vec{v}_2, \vec{v}_3, \vec{v}_4$ be the coordinates of one of the two vertices on each edge. Let $\vec{p} = (p_x, p_y)$ and r be the unknown position and radius of the shared incircle.

For the incircle to be tangent to all four edges simultaneously, the perpendicular distance from the center $\vec{p}$ to each edge i must be exactly equal to r. Using vector projection, we can express the distance to a line as:

$$\hat{n}_i \cdot (\vec{p} - \vec{v}_i) = r \quad \text{for } i \in \{1, 2, 3, 4\} \tag{10}$$

We can rearrange this equation to separate the unknown variables ($\vec{p}$ and r) from the variables defined by the graph's geometry ($\vec{v}_i$):

$$\hat{n}_i \cdot \vec{p} - r = \hat{n}_i \cdot \vec{v}_i \tag{11}$$

This gives us a system of 4 linear equations. We can represent this system in matrix form as $Q\vec{\xi} = \vec{w}$, where $\vec{\xi}$ contains our unknowns:

$$\underbrace{\begin{bmatrix} n_{1x} & n_{1y} & -1 \\ n_{2x} & n_{2y} & -1 \\ n_{3x} & n_{3y} & -1 \\ n_{4x} & n_{4y} & -1 \end{bmatrix}}_{Q} \underbrace{\begin{bmatrix} p_x \\ p_y \\ r \end{bmatrix}}_{\vec{\xi}} = \underbrace{\begin{bmatrix} \hat{n}_1 \cdot \vec{v}_1 \\ \hat{n}_2 \cdot \vec{v}_2 \\ \hat{n}_3 \cdot \vec{v}_3 \\ \hat{n}_4 \cdot \vec{v}_4 \end{bmatrix}}_{\vec{w}} \tag{12}$$

Notice that Q is a 4×3 matrix. We have 4 equations but only 3 unknowns (p_x, p_y, and r). For an overdetermined system to have a valid solution, the right-hand side vector $\vec{w}$ must lie perfectly within the column space of Q. We do not actually need to solve for $\vec{\xi}$. We only care that such an incircle exists. If we can eliminate $\vec{\xi}$ entirely, we are left with a relationship that the vertices $\vec{v}_i$ must satisfy. Recall that a vector $\vec{w}$ is in the column space of Q if and only if it is orthogonal to the left null space of Q. Let $\vec{w}^T = \begin{bmatrix} w_1 & w_2 & w_3 & w_4 \end{bmatrix}$ be a basis vector for the left null space of Q, such that:

$$\vec{w}^T Q = \vec{0}^T \tag{13}$$

Because the normals $\hat{n}_i$ are strictly fixed constants in the 22.5° system, the matrix Q is constant. Therefore, $\vec{w}$ is also constant and can be precomputed.

For the incircle to exist, we must satisfy:

$$\vec{w}^T \vec{\xi} = w_1(\hat{n}_1 \cdot \vec{v}_1) + w_2(\hat{n}_2 \cdot \vec{v}_2) + w_3(\hat{n}_3 \cdot \vec{v}_3) + w_4(\hat{n}_4 \cdot \vec{v}_4) = 0 \tag{14}$$

By expanding the inner dot products $\hat{n}_i \cdot \vec{v}_i = n_{ix}x_i + n_{iy}y_i$, we group the terms by the coordinate variables of our vertices:

$$\sum_{i=1}^{4} ((w_i n_{ix})x_i + (w_i n_{iy})y_i) = 0 \tag{15}$$

Note that this is a strictly linear, homogeneous constraint acting entirely on the node positions x_i and y_i, which can also be expressed as a row in the constraint matrix A without ever solving for the position or radius of the incircle.

In general, constraining a quadruplet (that contains at least one unconstrained edge) will remove one degree of freedom. However, due to internal symmetries across the tiling, it is possible for a single quadruplet constraint to collapse multiple straight skeleton vertices at once, while still only spending one degree of freedom. Therefore, we want to strategically select quadruplets that maximize the number of straight skeleton vertices they collapse.

But more importantly, it is possible for a quadruplet constraint to be satisfied but not actually reduce the number of vertices in the straight skeleton. This is because a vertex only appears in the straight skeleton if and only if it is strictly closer to its constituent edges than to any other edge in the polygon. Therefore, if another edge "invades" the incircle, the quadruplet constraint is effectively useless.

Even worse, it is possible for the enforcing of a quadruplet constraint to cause an edge to collapse to 0 or negative length, violating the planarity of the tiling. These two effects are defined by inequalities. Therefore, we must resort to a Mixed-Integer Linear Programming (MILP) formulation to strategically select the optimal set of quadruplets that maximize the number of collapsed straight skeleton vertices while ensuring the physical foldability of the tiling.

4.3 Quadruplet selection via MILP

We formulate the selection of these quadruplets as a MILP problem. Let $z_k \in \{0, 1\}$ be the boolean decision variable to enforce quadruplet k. Our objective is to maximize geometric simplicity:

$$\max \sum_k W_k z_k \tag{16}$$

The weight W_k is heavily biased. Unresolved quadruplets containing concave (reflex) vertices create disproportionately messier crease patterns if not collapsed into a clean internal vertex. Therefore, quadruplets involving reflex vertices are assigned a priority weight ($W = 10.0$), while standard convex quadruplets receive a base weight ($W = 1.0$).

4.3.1 Continuous Variables and the Anti-Bowtie Clearance

The MILP must evaluate the continuous physical consequences of toggling $z_k = 1$. For every node u, we assign continuous variables x_u, y_u bounded by the grid dimensions $\pm N$. For every candidate quadruplet k, we assign continuous variables for its theoretical incircle center (Px_k, Py_k) and its radius $r_k \geq 0$.

If $z_k = 1$, the distance from the incircle center to all four edges in the quadruplet must exactly equal r_k. For an edge e with normal vector $\mathbf{n} = (n_x, n_y)$ and an origin node u, we enforce:

$$-C(1 - z_k) \leq \mathbf{n} \cdot (\mathbf{P}_k - \mathbf{p}_u) - r_k \leq C(1 - z_k) \tag{17}$$

where C is a sufficiently large Big-M scalar (in our architecture, $C_{dynamic} = 3N$) that mathematically disables the constraint when $z_k = 0$.

Furthermore, moving nodes to satisfy quadruplets risks causing the polygon to fold over itself—creating a non-physical "bowtie" intersection. To prevent this, we implement **Proximity Culling Clearance Constraints**. For every reflex vertex v_c and every opposing non-adjacent edge (u, v) with normal $\mathbf{n}$ and tangent $\mathbf{t}$, the reflex vertex must remain safely outside a minimum feature size tolerance, which we define as gap $= \frac{1}{4N}$.

We divide the space around the edge (u, v) into three boolean sectors: strictly inside the normal (b_1), strictly behind the start of the tangent (b_2), or strictly ahead of the end of the tangent (b_3). The reflex vertex must reside in at least one safe sector:

$$b_1 + b_2 + b_3 \geq 1 \tag{18}$$

$$\mathbf{n} \cdot (\mathbf{p}_c - \mathbf{p}_u) \geq \text{gap} - C(1 - b_1) \tag{19}$$

$$\mathbf{t} \cdot (\mathbf{p}_c - \mathbf{p}_u) \leq -\text{gap} + C(1 - b_2) \tag{20}$$

$$\mathbf{t} \cdot (\mathbf{p}_c - \mathbf{p}_v) \geq \text{gap} - C(1 - b_3) \tag{21}$$

By bounding these continuous variables, the MILP guarantees that any selected set of quadruplets $z_k = 1$ is physically realizable without causing 2D geometry collisions.

4.3.2 Iterative Integer Cuts for Database Diversity

A single topology can often be tiled in several geometrically distinct ways. To populate the database with diverse layouts from the same root topology, we loop the MILP solver. Upon finding an optimal set of active quadruplets $\mathcal{A}$, we log the configuration and inject a strict integer cut back into the solver:

$$\sum_{i \in \mathcal{A}} z_i \leq |\mathcal{A}| - d_{threshold} \tag{22}$$

where $d_{threshold} = 2$. This bans the previous solution and forces the MILP to swap at least two quadruplets, discovering alternative geometric variations.

[Figure Placeholder: A visual diagram comparing three distinct MILP tilings generated from the exact same unweighted topology. Highlight the specific quadruplets chosen in each iteration and how they radically alter the placement of the interior straight skeleton vertices.]

The solution to the MILP comes with a set of vertex positions that is guaranteed to be physically realizable (satisfying the constraints applied to M). However, it cannot guarantee that the tiling is fully locked. Therefore, we need to take the quadruplets selected by the MILP and apply them as additional rows in the constraint matrix A, then check if A has full rank. If not, we need to go back and select more quadruplets until we have a full-rank matrix that fully locks the tiling into a single crease pattern.

4.4 Quadruplet harvesting

The MILP maximizes the number of active quadruplets, but because an n-gon contains $\binom{n}{4}$ possible quadruplet combinations, it is computationally prohibitive to feed every possible quadruplet into the MILP.

Consequently, the quadruplets selected by the MILP might leave the 4D matrix rank-deficient (having fewer than $4n$ independent constraints), meaning some nodes can still "slide" freely along creases.

Before performing the expensive Gauss-Jordan elimination, we check the algebraic rank of the constraint matrix M. To do this instantaneously, we perform Gaussian Elimination over a finite field Z_p using a large prime (e.g., $P = 1000000007$).

If the matrix is rank-deficient, the Z_p elimination identifies exactly which columns (variables) lack pivots. We map these free variables back to their physical nodes, identifying the "sliding nodes." We then initialize a **Directed Depth-First Search (DFS) Scavenger**. The DFS explores the pool of unused, exhaustive quadruplet candidates, specifically filtering for quadruplets that involve the identified sliding nodes. It tentatively adds their 4D nullspace equations to M and re-evaluates the rank. Once the matrix achieves full rank (exactly $4n$), the DFS terminates.

Finally, the full-rank constraint matrix is passed to our custom C++ `Fraction` class routines, which execute exact Gauss-Jordan elimination to solve for the absolute 4D coordinates of the metric tiling.

This process is thorough, albeit roundabout. First we use heuristics to choose which quadruplets to consider first. Then we use MILP to select which of those quadruplets are the best, by applying constraints to the constraints themselves. If it turns out that the MILP selection is not sufficient,

we have to go back and scavenge through the unconsidered quadruplets to find which quadruplets would make A full rank. Only then do we actually apply the quadruplet constraints and solve for the vertex coordinates.

4.5 The Exact Linear Solver and the Two-Row Formulation

The MILP provides a guaranteed-safe set of geometric intents, but floating-point MILP coordinates are too noisy for strict origami raycasting. We must lock these intents into an exact fractional matrix in the 4D Hypercube basis $\mathbb{Q}^4$.

Consider a standard angular constraint requiring the edge between node u and node v to remain parallel to a known 22.5° directional vector $\mathbf{d} = (d_x, d_y)$. The 2D cross product must be exactly zero:

$$(v_x - u_x)d_y - (v_y - u_y)d_x = 0 \tag{23}$$

In our 4D basis, the physical 2D coordinates are derived from the 4D rational coordinates: $u = [x_1, y_1, z_1, w_1]^T$ and $v = [x_2, y_2, z_2, w_2]^T$. Substituting the hypercube projection mapping:

$$\Delta X = (x_2 - x_1) + \frac{\sqrt{2}}{2}((y_2 - y_1) - (w_2 - w_1)) = A_x + B_x\sqrt{2} \tag{24}$$

$$\Delta Y = (z_2 - z_1) + \frac{\sqrt{2}}{2}((y_2 - y_1) + (w_2 - w_1)) = A_y + B_y\sqrt{2} \tag{25}$$

The directional vector $\mathbf{d}$ is similarly defined as $d_x = a_x + b_x\sqrt{2}$ and $d_y = a_y + b_y\sqrt{2}$. Substituting these components into the cross product yields a polynomial expansion:

$$(A_x + B_x\sqrt{2})(a_y + b_y\sqrt{2}) - (A_y + B_y\sqrt{2})(a_x + b_x\sqrt{2}) = 0 \tag{26}$$

When the algebra is expanded and simplified, the variables naturally separate into two groups: those multiplied by 1, and those multiplied by $\sqrt{2}$. It takes the form:

$$C_{rational} + C_{irrational}\sqrt{2} = 0 \tag{27}$$

Crucially, because $\sqrt{2}$ is an irrational number, it is impossible for a non-zero rational number to cancel out a non-zero irrational number. Therefore, for the physical angle to be exactly zero, **both coefficients must independently evaluate to exactly zero**.

This is the core insight of the SEARCH 22.5 exact solver architecture: **Every single physical 2D geometric constraint generates exactly two independent rows in the 4D algebraic constraint matrix.**

$$\text{Row 1:} \quad C_{rational}(x_1, y_1, z_1, w_1, x_2, y_2, z_2, w_2) = 0 \tag{28}$$

$$\text{Row 2:} \quad C_{irrational}(x_1, y_1, z_1, w_1, x_2, y_2, z_2, w_2) = 0 \tag{29}$$

[Figure Placeholder: A visual matrix diagram showing the $Ax = b$ formulation. On the left, display a graph with a single highlighted edge. On the right, display the constraint matrix, explicitly

showing how the x, y, z, w columns of nodes u and v populate two distinct rows for that single physical edge.]

5 Database Generation Phase III: Crease Patterns and Folding

The exact tiling establishes a flawless 4D geometric foundation, locking the perimeter and the major structural nodes of the base. However, this abstract metric graph must now be routed into a physically flat-foldable Crease Pattern (CP) and subsequently folded into a layered 2D state to extract its structural tree.

5.1 The Float-to-Exact Straight Skeleton

To route the interior creases of the faces defined by the tiling, we utilize a straight skeleton algorithm. Because native exact-fraction straight skeleton algorithms are computationally prohibitive, we implement a hybrid "Float-to-Exact" mapping architecture.

First, we project the 4D exact vertices of a face into 2D floating-point Cartesian coordinates. To bypass the well-documented IEEE-754 singularity failures inherent to standard floating-point sweep-line algorithms, we apply an aggressive quantization filter, truncating coordinates to a fixed precision (`QUANTIZATION_DECIMALS = 3`) and deduplicating antiparallel spikes created by the MILP's vertex collapsing.

We feed this sanitized floating-point polygon into an off-the-shelf straight skeleton library to extract the *topological connectivity* of the interior creases. To reclaim exact mathematical precision, we map these floating-point nodes back to the $\mathbb{Q}^4$ hypercube grid. For boundary nodes, we search the exact 4D coordinate dictionary using a tight capture radius (`SNAP_TOLERANCE = 1e-3`). For internal nodes, we iteratively look at their resolved neighbors, extract the 22.5° directional angles of the incident creases, and perform an exact 4D algebraic intersection. This hybrid approach grants us the extreme speed of floating-point topological algorithms while guaranteeing the 100% rational geometric perfection of the final vertices.

5.2 Kawasaki Propagation and Hinge Routing

The resulting straight skeleton routes the paper efficiently, but it makes no guarantees regarding flat-foldability. In origami, Kawasaki's Theorem mandates that for a vertex to fold flat, the alternating sum of the angles between its incident creases must equal exactly 180° (or an alternating sum of 8, in our 22.5° integer units).

We observe that the straight skeleton inherently creates degree-3 vertices (which trivially violate Kawasaki's theorem) and other odd-degree intersections. To enforce flat-foldability, we developed a greedy Kawasaki-fixing algorithm utilizing recursive 4D raycasting.

The algorithm filters for odd-degree vertices that violate Kawasaki. For a target vertex, it evaluates two primary strategies:

1. **Hinge Addition:** It tentatively casts an auxiliary "hinge" ray outward along all 8 valid 22.5° directions.

2. **Hinge Removal:** It tentatively removes an existing hinge and recursively collapses any adjacent degree-3 vertices that are subsequently rendered redundant.

When a hinge ray is cast into a polygonal face, physical logic dictates it can only intersect the bounding edges of that specific face. We calculate the exact 4D intersection; if the ray strikes a boundary crease, it splits the edge, generates a new vertex, and reflects perfectly across the crease to propagate into the adjacent face.

Because one added hinge can cascade across the entire paper, we execute these tests on ultra-fast, shallow-copied clones of the CP. We evaluate each transformation using a strict cost metric: `(Remaining Odd Kawasaki Errors, Total Operations/Hinges Added)`. The algorithm permanently commits the transformation that resolves the local angular defect while minimizing the addition of new, cluttering creases.

[Figure Placeholder: A sequential diagram of Kawasaki Propagation. Frame 1: A degree-3 vertex in a straight skeleton. Frame 2: An auxiliary hinge ray is cast, splitting a neighboring edge. Frame 3: The ray reflects and propagates until it terminates at a boundary, leaving a fully valid, even-degree crease mesh.]

5.3 The Stacked Folded State Architecture

Once the CP is strictly valid, we algorithmically fold it into a mathematically flat 2D state. The CP explicitly tracks topological faces via Doubly-Connected Edge List (DCEL) logic, sorting half-edges counter-clockwise and discarding the infinite outer face using signed-area calculations .

To calculate the 2D folded coordinates, we construct a spanning tree of the faces via a Kinematics Breadth-First Search (BFS). Starting from an arbitrary root face, we walk the adjacency graph, recursively reflecting the exact 4D vertices of child faces across their connecting hinge edges.

However, calculating coordinates alone is insufficient; we must track the physical layering of the paper. We define a highly specialized data structure, `Fold225`, which separates abstract topological faces from physical paper layers (*instances*):

- `vertices`, `edges`, `faces`: The core geometric definitions in the folded 2D plane.
- `instances`: A list of stacks. Stack f corresponds to face f. If a face folds back onto itself multiple times, its stack will contain multiple *instances*.

Every instance possesses an implied address (f, i), where f is the face index and i is its depth in the stack. Within the stack, an instance is represented solely by its connections to other instances across its perimeter edges. For example, if instance (f_1, i_1) connects across edge e to instance (f_2, i_2), that address is logged in its connection array. Boundary edges simply terminate in a `Null` pointer. This pointer-based architecture completely models the non-manifold, layered reality of folded paper.

5.4 Orthogonal Slicing and Tree Extraction

With the paper successfully folded into a layered stack of 2D polygons, we must extract its uniaxial "Stick Figure" (the macroscopic tree). Because all layers overlap, we execute an orthogonal slicing

algorithm—conceptually identical to a medical CT scan—to isolate the connected appendages.

First, we determine the optimal projection axis $\mathbf{a}$ by evaluating the bounding box width along the primary orthogonal and diagonal vectors, selecting the axis that yields the maximum span. Every vertex $\mathbf{v}_i$ in the folded state projects a critical height onto this axis: $h_i = \mathbf{v}_i \cdot \mathbf{a}$. We isolate and sort the unique critical heights generated by the vertices .

We mathematically sweep a 1D slice line $\mathcal{L}(h)$ perpendicular to $\mathbf{a}$. Between any two adjacent critical heights (h_i, h_{i+1}), the topology of the folded layers is entirely immutable. Therefore, we sample the structure precisely at these critical heights.

When a slice intersects the folded state, it severs the topological faces. We utilize a highly optimized C++ routine (`split_and_rebuild_cpp`) to bifurcate the affected faces and instantly remap the multi-layered instance pointers to the newly severed geometry.

To extract the tree, we build an adjacency graph of the instances. Crucially, **we ignore any connections that occur across edges parallel to the slicing hinge** . By filtering these out, the instances cluster into discrete Connected Components, representing the physical "flaps" or "rivers" of the origami base flowing between the critical height slices.

Every connected component spanning between two slices forms an **Edge** in our macroscopic tree, with its length directly proportional to the physical axial distance $h_{i+1} - h_i$. Where these components converge and physically interact at a critical height slice, they form a **Node** in the tree. By collapsing and merging intermediate degree-2 nodes, the microscopic complexity of overlapping paper layers is flawlessly distilled into an abstract, undirected graph representing the physical appendages of the origami model .

[Figure Placeholder: The Slicing Algorithm. Show a folded base on the left with a 1D projection axis. In the center, show exploded views of the physical instance layers at three different slice heights. On the right, show the final extracted graph tree, visually mapping edges of the graph to the connected components (flaps) of the layers.]

6 Database Architecture and Infrastructure

To manage the generation of millions of crease patterns and their high-dimensional representations, SEARCH 22.5 implements a decoupled, highly parallelized database architecture utilizing SQLite, SQLAlchemy, and Python's `multiprocessing` library. The generation pipeline is strictly separated into two independent databases: the Topologies database and the Tilings database.

6.1 Phase I Storage: Topology Database

The Topology database (`topologies_N_symmetry.db`) acts as the initial repository for the Z3 SAT solver outputs. To manage the parallel workload efficiently, a `Prefix` table acts as a task queue, tracking which binary prefixes have been fully exhausted by the worker processes.

When a worker discovers a valid topology, the configuration of its internal edges is serialized into a highly compressed bit-string. To ensure uniqueness and provide a rapid indexing mechanism, this bit-string is hashed into a 64-bit integer using MurmurHash3 (`mmh3`). The topology is saved to the `State` table using a native `INSERT ... ON CONFLICT DO NOTHING` command. By shifting

the deduplication burden down to the SQLite C-engine via the `hashed_state` index, the Python worker processes can asynchronously write thousands of states per second without encountering race conditions or requiring expensive cross-process locking locks.

6.2 Phase II Storage: Tiling Database

The Tiling database (`tilings_N_symmetry.db`) is populated by an orchestrator script that pulls pending topologies from Phase I. Because the MILP quadruplet solver can generate multiple geometrically distinct tilings from a single topology via our diversity integer cuts, the Tiling database implements a one-to-many schema .

For each valid metric tiling successfully generated, canonicalized, and flat-folded, three key data structures are stored:

1. `hashed_tiling`: A 64-bit MurmurHash3 integer derived from the canonicalized exact 4D coordinate geometry. This prevents identically routed straight skeletons (which can occasionally emerge from different MILP branches) from duplicating in the database.

2. `tiling_blob`: A fully serialized (Pickled) dictionary containing the exact $\mathbb{Q}^4$ vertices, the exact edge connectivity, and the topological faces, allowing the crease pattern to be perfectly reconstructed downstream.

3. `embedding`: A dense 32-dimensional NumPy array (`float32` bytes) representing the tree's spectral fingerprint, optimizing the memory footprint for machine learning similarity searches.

7 Spectral Graph Theory and Tree Embedding

To map the extracted uniaxial stick-figure trees into a searchable continuous vector space, SEARCH 22.5 leverages Spectral Graph Theory.

7.1 Literature Review: The Graph Laplacian

Spectral graph theory analyzes the properties of a graph in relationship to the characteristic polynomial, eigenvalues, and eigenvectors of matrices associated with the graph, most notably the Adjacency matrix A and the Laplacian matrix L [2]. The Laplacian is defined as $L = D - A$, where D is the diagonal degree matrix.

The Laplacian matrix has been shown to hold immense representational power for graph topologies. Because L is a symmetric, positive semi-definite matrix, its eigenvalues are real and non-negative: $0 = \lambda_1 \leq \lambda_2 \leq \cdots \leq \lambda_n$. The multiplicity of the zero eigenvalue corresponds exactly to the number of connected components in the graph. For a connected graph (like an origami tree), $\lambda_1 = 0$ and $\lambda_2 > 0$.

This second smallest eigenvalue, λ_2, is famously known as the *algebraic connectivity* or the *Fiedler value*, introduced by Miroslav Fiedler in 1973 [3]. The Fiedler value dictates the global connectivity and structural bottlenecks of the network. Furthermore, eigendecompositions of the

Laplacian have been extensively and successfully utilized in computer vision and geometric hashing for graph-matching problems, as isomorphic graphs are strictly cospectral (sharing identical eigenvalues), while the Laplacian spectrum provides robust, graded continuous distances for non-isomorphic but structurally similar shapes [1].

7.2 The 32-Dimensional Fingerprint

Following this theoretical precedent, we compute the eigenvalues of the Laplacian matrix L for every extracted origami tree. The edge lengths of the tree (derived from the orthogonal slicing heights) are mapped to graph weights $w_{uv} = 1/L_{uv}$, such that short, tight flaps exhibit stronger algebraic connectivity bounds than long, sweeping branches .

The eigenvalues are calculated via standard numerical linear algebra (`np.linalg.eigvalsh`). Because all valid origami bases exist as a single connected component, the trivial $\lambda_1 = 0$ eigenvalue is systematically discarded. The remaining non-zero eigenvalues are sorted in ascending order. To ensure the database maintains a uniform mathematical dimensionality regardless of the complexity of the origami base, the sequence is either truncated or padded with trailing zeros to exactly 32 dimensions (`DIMENSION = 32`) . This 32-dimensional continuous vector serves as the immutable geometric fingerprint for the database.

8 Database Querying and Retrieval

The ultimate objective of SEARCH 22.5 is to allow a user to retrieve a complex, exact-fraction crease pattern in constant time. This is achieved via our Query Pipeline and the interactive graphical interface.

8.1 Ephemeral FAISS Indexing and NumPy Caching

A standard SQLite database is highly inefficient for high-dimensional vector similarity searches. Furthermore, standard indexing structures are rigid; altering the geometric importance of certain graph features would traditionally require mutating and re-saving the entire multi-gigabyte database.

To bypass this, we maintain a raw `.npy` (NumPy) cache of the database embeddings alongside an ordered mapping array . Loading a dense matrix directly into RAM from a binary `.npy` file is over 100 times faster than iterating through SQLite blobs. At query time, the system loads this matrix and constructs an ephemeral FAISS (Facebook AI Similarity Search) `IndexFlatL2` index entirely in memory. This allows the search algorithm to instantly scan millions of 32-dimensional vectors and retrieve the nearest L2 (Euclidean) matches in mere milliseconds.

8.2 Eigenvalue Weighting and Tuning

Raw Euclidean distance across the Laplacian spectrum is flawed for origami design. High eigenvalues ($\lambda \geq 2.0$) correspond to localized, high-frequency structures within the graph, such as minor topological forks or tiny flaps at the tips. Conversely, low eigenvalues ($\lambda \leq 0.5$) govern the global,

macroscopic branches (the spine, the major appendages). In origami design, matching the global proportions of the base is vastly more important than precisely mirroring the microscopic flaps.

Because our FAISS index is ephemeral, we can apply dynamic scaling weights to both the query vector and the entire database matrix instantaneously at query time . We implemented three distinct weighting architectures:

- **Value Decay:** Weights drop exponentially as the eigenvalue itself increases: $w_i = e^{-t \cdot \lambda_i}$.
- **Index Decay:** Weights drop strictly based on the dimensional sequence order: $w_i = e^{-t \cdot i}$.
- **Inverse:** An aggressive, physics-inspired harmonic penalty: $w_i = 1/(\lambda_i + \epsilon)$.

Empirical testing on thousands of interactive queries demonstrated that **Value Decay** with a parameter of $t \approx 0.03$ perfectly balances structural fidelity and local flexibility, heavily penalizing mismatches in the global Fiedler values while remaining highly tolerant of extra flaps at the tips of the branches.

8.3 Interactive GUI

To bridge the gap between human design and the hyperspace index, we built an interactive Matplotlib GUI. The user is presented with a blank canvas and can click and drag to draw a completely custom uniaxial stick figure .

The spatial coordinates of the drawn nodes define the Euclidean lengths of the physical flaps. When the query is executed, the GUI constructs a NetworkX graph, normalizes the edge lengths, extracts the Laplacian eigenvalues, applies the $t = 0.03$ value decay, and pings the FAISS engine. The result is a unified Megaplot that instantaneously presents the user with the mathematically perfect 22.5° crease patterns whose internal folded skeletons most closely map to their hand-drawn intent.

9 Computational Scaling and Architecture

The combinatorial explosion of valid grid topologies represents the primary computational bottleneck. Exhaustive profiling reveals the total number of states $\mathcal{N}(N)$ scales with the area of the grid:

$$\mathcal{N}(N) \sim \mathcal{O}(\gamma^{N^2}) \tag{30}$$

Empirical data establishes $\gamma \approx 5.5$. Utilizing Python multiprocessing paired with C++ `pybind11` bindings, a single state traverses the entire pipeline in approximately 0.4 seconds, requiring 0.04 KB of SQLite storage. While databases up to $N = 5$ synthesize effectively on standard workstation clusters, pushing the architecture to $N = 6$ and beyond moves directly into the domain of dedicated supercomputing.

9.1 Parallelization via Prefix Pruning

As grid area expands, the search space $O(2^{4N^2})$ rapidly exceeds the capacity of a single SAT solver instance. To solve this, we implemented a massive multiprocessing architecture utilizing *prefix seeding*.

The internal edges are given a strict, deterministic ordering. We can divide the search space by hardcoding the first k edges to a specific binary prefix (e.g., '10110...'), leaving the solver to find valid configurations for the remaining $(4N^2 - 2N) - k$ edges.

However, naively generating all 2^k prefixes results in millions of "dead" prefixes that inherently violate the planarity or degree constraints within their first few bits. To solve this efficiently, we execute a *Pre-Pruning Phase*. We spin up a miniature Z3 instance containing only the base geometric rules, and constrain its search exclusively to the first k variables. We iteratively extract valid models, blocking previously found prefixes by adding the logical negation of their exact bit-state back to the solver:

$$\text{Block Clause} = \bigvee_{i=1}^{k} (v_i \neq b_i) \tag{31}$$

This drastically shrinks the search tree. For example, an 18-bit prefix sequence natively contains 262,144 combinations, but Z3 dynamically pre-prunes this down to merely ∼40,000 viable prefixes.

These viable prefixes are stored in a centralized SQLite database queue. Parallel worker processes fetch a prefix, initialize a fresh Z3 solver with the prefix hardcoded as absolute truth, and exhaustively discover every valid topology that exists down that specific branch of the combinatorial tree.

9.2 Canonicalization and Database Storage

Because the SAT solver inherently generates isomorphically identical graphs rotated or reflected across the paper, we canonicalize the geometries before saving.

We apply all 8 rigid transformations of the D_4 group to the active edge set. The edges are strictly sorted, and the transformation that yields the lexicographically smallest edge tuple is selected as the canonical absolute representation of that topology.

To minimize disk I/O and storage overhead, the canonical edge list is compressed into a contiguous binary byte-string, representing the truth states of the ordered edges. A 64-bit MurmurHash3 (`mmh3`) integer is generated from this byte-string. The worker flushes these states in batches of 50 to an SQLite writer process, which utilizes a native 'INSERT ... ON CONFLICT DO NOTHING' statement bound to the 64-bit hash index. This pushes the deduplication burden down to the C-level of the database engine, rendering it lightning-fast, immune to uncommitted batch duplicates, and highly resilient to process interruptions.

10 Conclusion

SEARCH 22.5 fundamentally reframes the computational origami design flow. By shifting the computational burden of integer programming and exact Gaussian elimination to an exhaustive

precomputation phase, we enable instantaneous, constant-time design retrieval. The utilization of the 4D Hypercube basis natively prevents floating-point inaccuracies, while our orthogonal slicing and spectral embedding techniques establish a robust bridge between abstract user intent (trees) and exact physical realizations (tiled crease patterns).

database scaling: how big of a database is necessary for good results, and how much training time and database memory size is required

quality of results

References

[1] Nair Maria Maia de Abreu. "Old and new results on algebraic connectivity of graphs". In: *Linear Algebra and its Applications* 423.1 (2007), pp. 53–73.

[2] Fan R. K. Chung. *Spectral Graph Theory*. Vol. 92. American Mathematical Society, 1997.

[3] Miroslav Fiedler. "Algebraic connectivity of graphs". In: *Czechoslovak Mathematical Journal* 23.2 (1973), pp. 298–305.